

PRODUCT REQUIREMENTS DOCUMENT

Unified File Operations Platform

Cross-Platform File Manager, Transfer, Sync & Governance

Definitive PRD V5 — Complete Merged Specification

Field	Value
Version	V5 (Definitive)
Status	Product Requirements Draft
Date	March 13, 2026
Classification	Confidential
Audience	Product, Design, Engineering, Security, Platform, GTM
Timeline	18–24 Months
Model	Free Individual / Paid Business
Tech Stack	Tauri 2 + Rust + React/Vite (Desktop) + Next.js (Admin Console)
Key Update	V4.1 tech stack revision integrated: Vite replaces Next.js static export, TanStack Suite, React Aria

Table of Contents

Table of Contents.....	2
1. Executive Summary	6
1.1 Competitive Landscape.....	6
2. Product Vision.....	8
3. Product Thesis	8
4. Problem Statement	8
5. Product Goals	10
5.1 Primary Goals.....	10
5.2 Secondary Goals.....	10
5.3 Quantified Success Targets	10
6. Non-Goals.....	11
7. Product Principles	11
7.1 Core Principles	11
7.2 Simplicity Principles.....	11
8. Target Users	13
8.1 General Users	13
8.2 Power Users / Developers / Creatives	13
8.3 IT / Enterprise / MSP / Media Ops.....	13
8.4 Detailed Persona Matrix	13
9. User Modes.....	15
9.1 Simple Mode (Default).....	15
9.2 Advanced Mode	15
9.3 Organization-Managed Mode.....	15
10. Packaging, Account Model, and Monetization	16
10.1 Product Packaging	16
10.2 Account Model.....	16
10.3 Monetization	16
10.4 Feature-by-Tier Comparison	16
11. Platform and Technical Stack	18
11.1 Architecture Overview	18
11.2 Desktop Frontend: Why React + Vite, Not Next.js Static Export.....	20
11.3 TanStack Suite: Table, Virtual, Query	21
11.3.1 TanStack Table.....	21
11.3.2 TanStack Virtual	21

11.3.3 TanStack Query.....	21
11.4 React Aria for Complex Accessibility.....	22
11.5 Architectural Principles.....	23
11.6 Key Rust Crates	23
11.7 Key Frontend Libraries.....	23
11.8 Why This Stack	24
11.9 Shared Design System Architecture	24
12. Product Scope Overview	26
13. Functional Requirements — File Management Engine	27
13.1 Core Browsing.....	27
13.2 Views.....	27
13.3 Core Operations	27
13.4 Search.....	27
13.5 Batch Rename.....	28
13.6 Preview.....	28
13.7 Archive Tools.....	28
13.8 Integrity and Organization	28
13.9 Feature Priority Matrix.....	28
14. Functional Requirements — Transfer Orchestration Engine	30
14.1 Required Connectors at GA	30
14.2 Connector Detail Matrix.....	30
14.3 Explicit Transfer Scope	31
14.4 Transfer Features.....	31
14.5 Drive-to-Drive and Peer-Transfer Behavior.....	32
14.6 Server-to-Server Direct Transfer	32
14.7 Connection Management	32
14.8 Device Discovery.....	32
14.9 Migration Workflows	32
15. Functional Requirements — Sync & State Engine	33
15.1 Sync Modes.....	33
15.2 Execution Modes.....	33
15.3 Dry-Run and Preview	33
15.4 Conflict Philosophy.....	33
15.5 Verification and Rollback.....	33
15.6 Sync Contexts	33
15.7 Sync Feature Priority.....	33

16. Functional Requirements — Universal Naming Compatibility Engine	35
16.1 Why This Engine Is Required.....	35
16.2 Required Domains.....	35
16.3 Logical vs Physical Naming.....	35
16.4 Intervention Tiers.....	35
16.5 Policy Profiles.....	36
16.6 Mapping and Restoration	36
16.7 Edge Cases.....	36
16.8 Engine Architecture	36
17. Functional Requirements — Governance & Enterprise Control Plane	38
17.1 RBAC	38
17.2 Policy Engine.....	38
17.3 Approvals	38
17.4 Audit Logs	38
17.5 Device Health Dashboard	38
17.6 Shared Workspaces.....	38
18. Functional Requirements — Security, Privacy, and Encryption.....	39
18.1 Desktop Security	39
18.2 Credentials	39
18.3 Encryption	39
18.4 Session Controls	39
19. Functional Requirements — API and CLI	40
19.1 CLI.....	40
19.2 API.....	40
20. Functional Requirements — AI Assistance & Automation Engine.....	41
20.1 Scope	41
20.2 Use Cases.....	41
20.3 Safety Rules	41
20.4 Privacy.....	41
21. User Experience and Interaction Requirements	42
21.1 Information Architecture	42
Simple Mode.....	42
Advanced Mode Adds.....	42
Admin Console	42
21.2 Onboarding.....	42
21.3 Guided Flows	42

21.4 Content Design Rules	42
21.5 Confidence and Recovery	42
21.6 Key UI Components	42
22. Key User Journeys.....	44
22.1 Local Folder to OneDrive with Incompatible Names	44
22.2 macOS to Windows Share Two-Way Sync	44
22.3 Enterprise-Governed External Transfer	44
22.4 AI-Assisted Cleanup.....	44
22.5 Drive-to-Drive on Same Computer	44
22.6 Computer-to-Computer over LAN	45
22.7 Computer Migration.....	45
22.8 External Drive Backup.....	45
23. Non-Functional Requirements	46
23.1 Qualitative	46
23.2 Quantified Targets.....	46
24. Success Metrics.....	47
24.1 Categories	47
24.2 Quantified KPI Targets.....	47
25. Risks and Mitigations	48
26. Engineering Decomposition	49
26.1 Rust Core Modules.....	49
26.2 Desktop UI Modules.....	49
26.3 Admin Console Modules	49
27. Launch Definition	51
27.1 GA Requirements.....	51
27.2 Internal Sequencing	51
28. Development Timeline — 18–24 Months.....	52
28.2 Milestones	52
29. Team Composition.....	53
30. Final Product Definition.....	54
31. External Constraints.....	55
32. Post-v1 Roadmap	56
Appendix A: Glossary	57

1. Executive Summary

This product is a unified file operations platform for individuals and enterprises. It combines a premium cross-platform desktop file manager with enterprise-grade transfer, synchronization, compatibility handling, governance, and AI-assisted automation.

The product direction takes inspiration from platforms such as ForkLift, which combine dual-pane file management, remote connections, preview, sync, and file transfer workflows in one experience. ForkLift's current product and manual positioning confirm the value of a single product that bridges local files, remote servers, and cloud services.

The defining product difference is not only “file manager + transfer client.” It is a compatibility-first operations engine designed to keep transfers and sync jobs moving across Windows, macOS, Linux, NAS, servers, removable drives, other computers, and cloud providers even when naming conventions, path rules, case sensitivity, Unicode behavior, and provider restrictions do not match. Windows still has filesystem-specific naming rules, APFS can be case-sensitive, ext4 still has a 255-byte filename limit, and OneDrive/SharePoint enforce naming and path restrictions.

The product must also be easy for general users. That is a top-level requirement, not a design preference. The platform must be consumer-simple by default, power-user fast when needed, and enterprise-governed where required.

Key Value Proposition

One platform. Every OS. Every protocol. Compatibility-first operations that keep work moving. Free for individuals. Enterprise-governed where required. AI-assisted from day one.

1.1 Competitive Landscape

The file management and transfer market is fragmented by platform, with no single tool delivering a complete cross-platform experience.

Feature	This Product	ForkLift 4	Cyberduck	WinSCP	FileZilla	Transmit 5
Platforms	macOS, Win, Linux	macOS only	macOS, Win	Windows only	macOS, Win, Linux	macOS only
Dual-Pane Manager	✓ Full	✓ Full	✗ No	✓ Basic	✗ No	✗ No
Cloud Integration	✓ S3, GDrive, Dropbox, OneDrive, B2, Azure	✓ Multiple	✓ Multiple	✓ S3 only	✗ No	✓ Multiple
FTP/SFTP/WebDAV	✓ All + SMB + NFS	✓ All	✓ All	✓ FTP/SFTP	✓ FTP/SFTP	✓ All

Bidirectional Sync	✓ Full engine	✓ Basic	✗ No	✓ Basic	✗ No	✗ No
Naming Compatibility Engine	✓ Multi-domain	✗ No	✗ No	✗ No	✗ No	✗ No
Drive-to-Drive / Peer Transfer	✓ First-class	✓ Basic	✗ No	✗ No	✗ No	✗ No
Enterprise Governance	✓ Full (RBAC, policy, audit)	✗ No	✗ No	✗ No	✗ No	✗ No
AI Assistance	✓ Yes	✗ No	✗ No	✗ No	✗ No	✗ No
API & CLI	✓ Both	✗ No	✓ CLI	✓ CLI	✓ CLI	✗ No
Encryption (E2E)	✓ Yes	✗ No	✓ Cryptomator	✓ AES	✗ No	✗ No
Price (Individual)	Free	\$29.95	Free (donate)	Free	Free	\$45

2. Product Vision

Build the best cross-platform file operations platform for Windows, macOS, and Linux by combining:

- easy-to-use file browsing and organization for general users
- fast dual-pane workflows for power users
- resilient transfer and sync across local, drive-to-drive, LAN, device-to-device, server, and cloud locations
- compatibility-aware naming and path handling
- enterprise controls including policy, approvals, and audit
- AI assistance that simplifies complex workflows instead of adding more complexity

3. Product Thesis

Most file tools treat naming conflicts, path-length issues, case mismatches, drive migration friction, and cloud restrictions as edge-case failures.

This product treats them as a core systems problem.

The platform must:

- keep jobs moving
- avoid unnecessary full-job failures
- preserve original logical filenames wherever possible
- make automatic compatibility interventions auditable
- explain issues in plain language
- restore original naming when files return to a compatible destination
- treat computer-to-computer transfer and drive-to-drive transfer on the same computer as first-class workflows, not hidden edge cases

4. Problem Statement

Users and teams work across:

- Windows
- macOS
- Linux
- internal SSDs and HDDs
- external USB and Thunderbolt drives
- SMB/NFS shares
- SFTP/FTP/WebDAV servers

- S3-compatible storage
- Dropbox
- Google Drive
- OneDrive
- Backblaze B2
- other computers on the same local network
- remote computers over secure protocols

Current workflows are fragmented and fragile:

- one tool for local browsing
- one tool for remote transfer
- another for cloud sync
- manual workarounds for filename issues
- poor visibility into why jobs failed
- too much technical jargon for normal users
- too little control for enterprise operations
- inconsistent handling of local drive migration
- unreliable or awkward computer-to-computer movement

This becomes especially painful when source and destination rules differ. Windows naming rules and reserved names differ from other systems, OneDrive/SharePoint enforce their own restrictions, APFS may be case-sensitive, and ext4 limits filenames to 255 bytes.

5. Product Goals

5.1 Primary Goals

1. Deliver a premium desktop file manager for Windows, macOS, and Linux.
2. Deliver enterprise-grade transfer and synchronization from day one.
3. Ship all listed connectors as GA in v1.
4. Prevent naming and path incompatibilities from unnecessarily stopping transfers or sync jobs.
5. Support both personal/local-first usage and tenant-based enterprise operation.
6. Deliver AI assistant and AI automation in v1.
7. Deliver a separate admin web console for policy, audit, device health, and approvals.
8. Make the product easy enough for general users without removing expert capability.
9. Treat computer-to-computer transfer, local drive-to-drive transfer, external-drive transfer, and guided device migration as core use cases.

5.2 Secondary Goals

10. Make the desktop app feel fast and near-native.
11. Hide complexity until it is needed.
12. Expose automation through API and CLI.
13. Make all important actions understandable, recoverable, and auditable.

5.3 Quantified Success Targets

Goal	Description	Success Metric
Cross-Platform Parity	Identical feature set and UX on macOS, Windows, and Linux	100% feature parity at v1 launch
Zero Filename Friction	Files transfer/sync without naming conflicts across any OS	<0.01% naming-related transfer failures
Ease of Use	General users perform core tasks without documentation	80%+ task completion in usability testing
Protocol Universality	All listed protocols and cloud providers operational at launch	14+ connectors GA
Performance	Handle large-scale transfers with minimal resource usage	<100MB RAM baseline, <2s cold start
Community Growth	Build active user base through free-for-individual model	50K users within 6 months of launch
Enterprise Adoption	Active managed teams using governance features	100 business teams within 6 months
AI Effectiveness	AI reduces manual work for common tasks	30%+ reduction in manual conflict handling

6. Non-Goals

For v1, the product is not trying to be:

- a document collaboration suite
- a file-editing platform
- a full backup imaging / bare-metal restore product
- a media asset management suite
- a plugin marketplace
- a custom OS shell replacement

Why Non-Goals Matter

Explicitly stating what the product is NOT prevents scope creep and aligns the entire team on where to focus. Features in the non-goals list may become goals in future versions, but they must not dilute v1 delivery.

7. Product Principles

7.1 Core Principles

- 1. Simple first, powerful second.** A new user must be able to perform common tasks immediately without training.
- 2. Progressive disclosure.** Advanced controls should appear when needed, not dominate the default experience.
- 3. Compatibility without panic.** The app must handle cross-platform and provider-specific incompatibilities gracefully.
- 4. No unnecessary job failure.** One problematic filename should not collapse an otherwise valid transfer job.
- 5. Plain-language product writing.** The app explains issues in human language first and technical detail second.
- 6. Logical identity matters.** Preserve original names and provenance, even when destination-safe names must differ.
- 7. Local-first where possible, governed where necessary.** Personal users get a strong local experience; organizations get centralized control.
- 8. Transparency over magic.** Automatic handling is good, but users must still be able to see what happened.
- 9. Local and peer transfer are first-class.** Moving files from one drive to another, from one computer to another, or from a local device to removable storage must get the same quality of queueing, verification, compatibility handling, and reporting as cloud/server workflows.

7.2 Simplicity Principles

14. A general user should be able to browse, connect a common cloud provider, transfer files, create a sync job, copy files to another drive, and move files to another computer without documentation.
15. The default interface should not look like a sysadmin console.
16. The app should use phrases like “We adjusted 3 names so the transfer could continue,” not “Destination normalization policy applied.”
17. All destructive or confusing operations must have clear explanation and recovery guidance.
18. Default settings should be safe, recommended, and low-friction.

8. Target Users

8.1 General Users

Need:

- easy browsing
- simple transfer and sync
- cloud access
- understandable warnings
- confidence and recovery
- easy drive-to-drive copying
- easy transfer to external drives
- easy movement between two computers

8.2 Power Users / Developers / Creatives

Need:

- dual-pane speed
- tabs and workspaces
- advanced rename
- search and preview
- checksums
- archives
- remote operations
- server-to-server transfers
- precise drive-to-drive and device-to-device workflows

8.3 IT / Enterprise / MSP / Media Ops

Need:

- policy controls
- audit logs
- RBAC
- approvals
- health dashboards
- device visibility
- encryption controls
- API and CLI
- consistent behavior across mixed environments
- predictable migration and endpoint-to-endpoint operations

8.4 Detailed Persona Matrix

Persona	Description	Key Needs	Success Criteria
Alex (General User)	Non-technical user managing personal files across devices, cloud, backups	Drag-and-drop, visual sync status, plain-language errors, one-click cloud backup, easy drive copy	Completes first transfer without documentation
Sam (Developer)	Full-stack dev, sysadmin managing servers and deploying code	SSH/SFTP, terminal, batch ops, scriptable workflows, Git-friendly naming, API/CLI	Sets up SFTP + sync in <5 minutes
Maya (Creative)	Designer/photographer with large files across Mac and Windows	Fast large-file transfers, preview, batch rename with patterns, reliable project sync	Syncs 50GB project folder without naming failures
Jordan (IT Admin)	IT department managing file infrastructure and compliance	Centralized config, audit logs, SSO, policy enforcement, encrypted transfers, device health	Rolls out to 100 users with policy controls in one day
Pat (Small Biz Owner)	Freelancer/small team needing simple cloud backup	Set-and-forget sync, cost-effective, works across mixed OS environments	Creates automated backup sync in Simple mode

9. User Modes

9.1 Simple Mode (Default)

This is the default mode. It emphasizes:

- clean layout
- fewer visible controls
- common tasks first
- guided setup flows
- plain-language prompts
- recommended defaults
- minimal jargon
- common cloud connections surfaced first
- simple drive copy and computer transfer flows

9.2 Advanced Mode

This exposes:

- full dual-pane power workflows
- detailed metadata
- regex and token-based rename
- transfer tuning
- checksum options
- detailed sync policy controls
- scripting and automation controls
- compatibility inspection tools
- low-level connection options

9.3 Organization-Managed Mode

This overlays policy and governance:

- enterprise login and tenant context
- visible restrictions and approvals
- policy-driven defaults
- shared connections and workspaces
- compliance-aware messaging

Mode Implementation Requirement

The product must support: Simple mode as the default, Advanced mode as an optional user setting, Organization-managed overlays without making the base UI feel enterprise-heavy.

10. Packaging, Account Model, and Monetization

10.1 Product Packaging

- Desktop app for daily file operations
- Admin web console for policy, audit, health, approvals, and org management

10.2 Account Model

- Personal mode: local-first, account optional for core local use
- Organization mode: tenant/account required for governed usage

10.3 Monetization

- Personal: free for all features (individual use)
- Business: subscription for team/admin/governance capabilities

10.4 Feature-by-Tier Comparison

Feature	Free (Individual)	Business (\$8/user/mo)	Enterprise (Custom)
Desktop file manager (full)	✓	✓	✓
All transfer protocols	✓	✓	✓
All cloud integrations	✓	✓	✓
Sync engine (all modes)	✓	✓	✓
Naming compatibility engine	✓	✓	✓
Built-in terminal	✓	✓	✓
Encryption	✓	✓	✓
AI assistant & automation	✓	✓	✓
API & CLI access	✓	✓	✓
Drive-to-drive & peer transfer	✓	✓	✓
Migration workflows	✓	✓	✓
Centralized team management	✗	✓ Admin console	✓ Full
Shared connections & workspaces	✗	✓	✓
RBAC (roles & permissions)	✗	✓ Standard roles	✓ Custom roles
Audit logging	✗	✓ 90-day retention	✓ Custom retention

Policy engine	X	✓ Standard policies	✓ Custom policies
Approval workflows	X	✓	✓
Device health dashboard	X	✓	✓
SSO / SAML / SCIM	X	✓ SAML 2.0	✓ Full + custom IdP
Priority support	Community forum	Email + chat (24hr SLA)	Dedicated CSM + SLA
Custom branding	X	X	✓
On-premise deployment	X	X	✓
AI governance controls	X	✓	✓
SIEM integration	X	X	✓

Monetization Philosophy

Individual users get everything—no artificial limits, no paywalls, no ads. Business features (team management, audit, SSO, compliance, governance) are the revenue driver. Happy individual users become business advocates.

11. Platform and Technical Stack

11.1 Architecture Overview

The platform uses a layered architecture with strict separation between the privileged Rust core and the UI layer. The desktop app must not depend on a server-rendered UI runtime for core operation. All privileged logic lives in Rust.

Layer	Technology	Purpose
Application Shell	Tauri 2.0	Cross-platform windowing, native OS integration, system tray, auto-update, IPC bridge between Rust and frontend
Core Engine	Rust (tokio async runtime)	All privileged logic: file ops, transfer engine, sync engine, compatibility engine, encryption, connectors, audit, scheduler
Desktop UI	React + TypeScript + Vite	Dual-pane file browser, transfer center, sync designer, AI assistant, guided wizards, settings
File List / Data Grid	TanStack Table	Headless table primitives: sorting, column resize, row selection, custom cell rendering, keyboard navigation
Directory Virtualization	TanStack Virtual	Virtualized row rendering for directories with 10K+ files. Only visible rows in DOM.
Server State	TanStack Query	Async data fetching, caching, and synchronization between React frontend and Rust backend via Tauri IPC
Client State	Zustand	Lightweight client-side state: UI state, pane layout, tab state, modal state, user preferences
Styling	Tailwind CSS + shadcn/ui	Utility-first CSS with accessible component library built on Radix UI primitives
Complex Accessibility	React Aria (Adobe)	Headless accessibility hooks for custom controls: multi-select file list, tree view, drag-and-drop, focus management
Terminal Emulator	xterm.js + Rust PTY backend	Embedded terminal for local and remote shell sessions
Local Database	SQLite (via rusqlite)	Configuration, transfer history, sync state, bookmarks, compatibility mappings, audit events
Admin Console	Next.js (full) + TypeScript	Server-rendered web app: policy management, RBAC, device health, audit explorer, approvals, AI governance, billing
Shared Design System	Shared tokens + shadcn/ui	Common color tokens, spacing, typography, and component APIs shared between desktop app and admin console

Packaging	Tauri bundler	Native installers: .dmg (macOS), .msi/.exe (Windows), .deb/.ApplImage (Linux)
-----------	---------------	---

11.2 Desktop Frontend: Why React + Vite, Not Next.js Static Export

Key Decision: Desktop UI uses Vite, not Next.js

The original V4 PRD specified Next.js with static export for the desktop frontend. This has been revised to React + TypeScript + Vite. The admin console retains full Next.js, where server-side capabilities are genuinely valuable.

The revision is based on five technical realities:

- 1. Tauri's frontend is a webview rendering static assets.** There is no server, no Node.js runtime inside the desktop shell. The frontend is purely client-side code. This is exactly what Vite is designed for.
- 2. Next.js static export disables its most valuable features.** Setting `output: 'export'` loses SSR, ISR, API routes, middleware, server components, and dynamic server-side routing. You carry the complexity of Next.js with none of its benefits.
- 3. Vite provides a materially better developer experience.** Dev server starts in <300ms (vs Next.js 2–5s). HMR is near-instant. No confusion about server vs client code. Build output is straightforward static assets.
- 4. Tauri recommends Vite as the primary React setup.** Tauri's `create-tauri-app` scaffolding generates a Vite project by default. Tauri + Vite is the most tested and supported path.
- 5. Next.js remains correct for the admin console.** SSR, API routes, middleware, and server components are genuinely valuable for a web-based admin console.

Capability	Vite (Desktop)	Next.js Static Export	Next.js Full (Admin)
Dev server start	<300ms	2–5 seconds	2–5s (acceptable for web)
SSR / Server Components	N/A (not needed)	Disabled by static export	✓ Available and valuable
API Routes	N/A (Rust handles API)	Disabled by static export	✓ Available for auth, webhooks
Config complexity	Minimal (vite.config.ts)	next.config.js + export workarounds	Standard Next.js config
Tauri integration	First-class (official scaffold)	Supported but secondary	N/A (web-only)
Client-side routing	React Router	Next.js router (client-only)	Next.js router (full)

11.3 TanStack Suite: Table, Virtual, Query

The file manager’s core UI is a data-intensive table. TanStack’s headless libraries provide the performance and flexibility required.

11.3.1 TanStack Table

A dual-pane file manager is fundamentally two interactive data tables. Each pane must support sortable columns, resizable columns, row selection (single/shift/ctrl/pattern), custom cell rendering (icons, badges, formatting), column visibility toggling per user mode, context menus per row, and full keyboard navigation. TanStack Table is headless, fully typed, and integrates cleanly with shadcn/ui and React Aria.

11.3.2 TanStack Virtual

The PRD requires 10K-file directory listing in <500ms. Standard React rendering of 10K DOM elements takes 500–800ms before interaction. TanStack Virtual renders only visible rows (~60), recycling as the user scrolls.

Metric	Without Virtualization	With TanStack Virtual
10K files: initial render	500–800ms (DOM-bound)	<50ms (~60 rows rendered)
10K files: scroll performance	Janky (all rows in DOM)	Smooth 60fps (row recycling)
100K files: feasible?	No (browser crashes)	Yes (same ~60 rows in DOM)
Memory (10K rows)	~50–100MB DOM overhead	~2–5MB DOM overhead

11.3.3 TanStack Query

Manages the async data layer between React frontend and Rust backend via Tauri IPC: automatic caching of directory listings, background refetching on file watcher events, optimistic updates for file operations, stale-while-revalidate for instant pane rendering, request deduplication, and retry with backoff.

11.4 React Aria for Complex Accessibility

shadcn/ui handles standard components (buttons, dialogs, menus). But a file manager has interaction patterns beyond standard UI:

Interaction Pattern	Challenge	React Aria Solution
Multi-select file list	ARIA grid role with row/range/toggle selection, screen reader announcements	useGridList: headless grid with full keyboard/selection support
Tree view (sidebar)	ARIA tree role with expand/collapse, keyboard nav, depth announcements	useTreeGrid: headless tree with correct ARIA roles
Drag and drop	Screen reader-accessible DnD, keyboard-accessible drag (Space/arrows)	useDrag, useDrop: accessible DnD with keyboard and live region announcements
Focus management	Correct focus between panes, tabs, sidebar, modals	useFocusManager, FocusScope: focus trapping and restoration
Breadcrumb navigation	ARIA navigation landmark with current location	useBreadcrumbs: headless breadcrumb with correct ARIA roles
Status announcements	Live regions for transfer progress, sync status, errors	ARIA live regions for dynamic content updates

How React Aria and shadcn/ui coexist: shadcn/ui provides the visual design system. React Aria provides headless accessibility hooks for custom file manager controls. React Aria handles behavior; shadcn/ui handles appearance. No conflicts.

11.5 Architectural Principles

- 1. Rust owns all privileged logic.** File ops, transfers, sync, encryption, credentials, and audit run exclusively in Rust. Frontend cannot bypass Rust. Enforced by Tauri's command allowlist.
- 2. Desktop frontend is purely client-side.** Vite-built React app runs entirely in Tauri webview. No server, no Node.js runtime, no SSR. All data from Rust via Tauri IPC.
- 3. Admin console is server-rendered.** Next.js (full) for policy management, audit, SSO. Server-side capabilities are genuinely valuable here.
- 4. Design system is shared.** Color tokens, spacing, typography defined once in a shared package. Both apps consume the same design tokens.
- 5. Accessibility is structural.** React Aria hooks integrated from day one into core file manager components. Not a Phase 7 audit fix.

11.6 Key Rust Crates

Crate	Layer	Purpose
tokio	Core	Async runtime for concurrent file transfers, network I/O, background workers
reqwest	Core	HTTP/HTTPS client for cloud APIs and WebDAV
ssh2 / async-ssh2-lite	Core	SSH/SFTP protocol implementation
suppaftp	Core	FTP/FTPS protocol implementation
rust-s3	Core	Amazon S3 and S3-compatible storage API
opendal	Core	Unified data access layer for 40+ storage services
unicode-normalization	Compat	NFC/NFD Unicode normalization for filename handling
notify	Sync	Cross-platform filesystem watcher for real-time sync
ring / aes-gcm	Security	Cryptographic primitives for encryption and checksums
rusqlite	Storage	SQLite for state persistence and compatibility mappings
serde / serde_json	Core	Serialization for config, IPC, API payloads
tauri-plugin-*	Shell	Tauri plugins (shell, dialog, fs, clipboard, notification, updater)
mdns-sd / zeroconf	Discovery	Local network device discovery for peer transfer
clap	CLI	Command-line argument parsing
indicatif	CLI	Progress bars for CLI operations

11.7 Key Frontend Libraries

Library	Version	App	Purpose
React	19.x	Both	UI framework with concurrent features and suspense

TypeScript	5.x	Both	Type safety across entire frontend
Vite	6.x	Desktop	Build tool and dev server for Tauri desktop frontend
Next.js	15.x+	Admin	Full-stack web framework for admin console
TanStack Table	8.x	Desktop	Headless table: sorting, column resize, selection, custom rendering
TanStack Virtual	3.x	Desktop	Virtualized rendering for 10K+ file directories
TanStack Query	5.x	Both	Async data fetching, caching, background refresh, optimistic updates
Zustand	5.x	Both	Lightweight client-side state management
Tailwind CSS	4.x	Both	Utility-first CSS framework
shadcn/ui	latest	Both	Accessible component library on Radix UI primitives
React Aria	3.x	Desktop	Headless accessibility hooks for file list, tree, DnD, focus
React Router	7.x	Desktop	Client-side routing (replaces Next.js router in desktop)
xterm.js	5.x	Desktop	Terminal emulator for embedded shell sessions
Lucide React	latest	Both	Icon library (shadcn/ui ecosystem)

11.8 Why This Stack

- **Rust OsString/OsStr:** Handles all OS-specific filename encodings natively. Foundation of the compatibility engine.
- **Tauri 2.0 vs Electron:** ~10x smaller binary (~15MB vs ~150MB), ~5x lower RAM, native webview. Mobile support for future.
- **Vite vs Next.js static export:** Faster dev server (<300ms vs 2–5s), simpler config, no SSR confusion, Tauri's recommended setup.
- **TanStack Table:** File manager IS a table. Headless primitives for sort, resize, select, keyboard without building from scratch.
- **TanStack Virtual:** 10K+ directories need virtualized rendering. Only ~60 DOM elements regardless of directory size.
- **TanStack Query:** Async IPC to Rust cached, deduplicated, background-refreshed. Remote directories feel instant.
- **React Aria:** Accessible multi-select lists, tree views, drag-and-drop. Headless hooks that work with any design system.
- **shadcn/ui + Tailwind:** Standard UI components with consistent design across desktop and admin console.
- **Next.js for Admin:** SSR, API routes, middleware genuinely valuable for web admin console. Correct tool for context.

11.9 Shared Design System Architecture

Package	Contents	Consumers
@ufop/design-tokens	Color palette (light/dark/high-contrast), spacing scale, typography, border radii, shadows, z-index, animations	Desktop + Admin
@ufop/ui-components	shadcn/ui-based: Button, Input, Select, Dialog, Dropdown, Tabs, Toast, Tooltip, Badge, Card, Skeleton	Desktop + Admin
@ufop/file-components	FileListTable (TanStack Table+Virtual+React Aria), TreeView, PaneLayout, BreadcrumbNav, TransferPanel, CompatibilityBadge	Desktop only
@ufop/admin-components	PolicyEditor, AuditExplorer, DeviceHealthCard, ApprovalQueue, RBACManager, BillingDashboard	Admin only

Monorepo structure: All packages in a single monorepo (pnpm workspaces or Turborepo). Changes to design tokens propagate to both apps in one PR.

12. Product Scope Overview

The product consists of six primary subsystems:

- 19. File Management Engine
- 20. Transfer Orchestration Engine
- 21. Sync & State Engine
- 22. Universal Naming Compatibility Engine
- 23. Governance & Enterprise Control Plane
- 24. AI Assistance & Automation Engine

Subsystem	Primary Responsibility	Key Differentiator
File Management Engine	Local and remote browsing, operations, search, preview, rename, archives	Full dual-pane manager with three user modes
Transfer Orchestration Engine	Queue-based transfer across all connector types with verification	14+ connectors GA, drive-to-drive and peer transfer as first-class
Sync & State Engine	Bidirectional sync with conflict resolution and rollback	Versioned backup sync, dry-run preview, resumable state
Naming Compatibility Engine	Cross-platform filename/path normalization and mapping	Multi-domain compatibility with tiered intervention and reversible mapping
Governance & Enterprise	RBAC, policy, approvals, audit, device health, admin console	Separate admin console with full policy engine
AI Assistance & Automation	Plain-language help, job suggestions, cleanup, automation	AI simplifies product instead of adding complexity

13. Functional Requirements — File Management Engine

13.1 Core Browsing

The desktop app must support:

- dual-pane navigation
- multiple tabs per pane
- tree view
- breadcrumb navigation
- multiple windows
- saved workspaces
- recent locations
- favorites and bookmarks
- mounted internal and external drives as first-class locations
- network computers and network shares as discoverable or connectable locations where supported

13.2 Views

- list view
- detail/column view
- grid/icon view
- compact view
- preview pane

13.3 Core Operations

Users must be able to:

- copy
- move
- rename
- duplicate
- delete
- create folders
- create files from templates
- drag and drop between panes, tabs, and locations
- queue multi-item actions
- copy or move between two drives on the same computer
- copy or move between local storage and removable media

13.4 Search

- filename search
- extension search
- size/date filters
- saved searches
- indexed content search where enabled
- remote search where supported
- search across multiple saved locations

13.5 Batch Rename

- token-based rename
- numbering
- case transformation
- extension updates
- date insertion
- pattern-based rename
- regex rename in Advanced mode

13.6 Preview

Safe preview for: images, PDFs, text/code, archive contents, metadata, audio/video where feasible, office files through safe rendering. Preview must be isolated and sandboxed.

13.7 Archive Tools

- browse archive contents
- extract archives
- create archives
- password-protected archive workflows where supported

13.8 Integrity and Organization

- checksum creation and verification
- duplicate detection
- tags
- labels
- color markers
- smart folders or saved filters

General-User Requirement

A general user must be able to complete common browse, move, rename, and search tasks without entering Advanced mode.

13.9 Feature Priority Matrix

Feature	Description	Priority
Dual-Pane Layout	Side-by-side browsing with independent navigation, resizable panes	P0
Drag & Drop	Between panes, to/from desktop and native file managers	P0
Tabbed Browsing	Multiple tabs per pane, drag to reorder, pin, restore session	P0
Quick Look / Preview	Inline preview for images, text, PDF, video, audio, code with syntax highlighting	P0
Batch Rename	Token-based, regex, numbering, date, case change, live preview	P0
Favorites & Bookmarks	Pin local/remote/cloud paths with drag-to-organize sidebar	P0
Search & Filter	Real-time filter + deep recursive search with regex, type, size, date	P0
All File Operations	Copy, move, delete, create, compress (zip/tar/gz/7z), extract	P0
Multi-Select	Click, Shift, Cmd/Ctrl, select all, invert, pattern	P0
Dark/Light Theme	System-aware with manual override, high-contrast accessibility	P0
Context Menus	Open With, Copy Path, Reveal in Native Explorer, custom actions	P0
Keyboard Navigation	Full keyboard nav, customizable shortcuts, optional Vim keybindings	P1
Command Palette	Cmd+K/Ctrl+K quick action bar for any action/setting/navigation	P1

14. Functional Requirements — Transfer Orchestration Engine

14.1 Required Connectors at GA

All of the following must be GA at launch:

- local filesystem
- LAN transfer
- SFTP
- FTP
- FTPS
- WebDAV
- SMB
- NFS
- S3-compatible object storage
- Dropbox
- Google Drive
- OneDrive
- Backblaze B2
- server-to-server direct transfer

14.2 Connector Detail Matrix

Connector	Auth Method	Key Features	Priority
Local Filesystem	OS permissions	Full CRUD, drive enumeration, removable media detection	P0
SFTP	Key / password / SSH agent	Jump hosts, custom port, keepalive, host key verify, resume	P0
FTP / FTPS	Password / TLS	Active/passive, explicit/implicit TLS, dir cache, resume	P0
WebDAV	Basic / digest / OAuth	HTTPS, locking, Nextcloud/ownCloud compatible	P0
SMB / CIFS	Password / Kerberos	Windows shares, network discovery, NAS, credential manager	P0
NFS	System auth	NFSv3/v4, Unix/Linux network shares	P0
Amazon S3	Access Key / IAM / SSO	Multipart upload, bucket mgmt, presigned URLs, S3-compatible	P0
Google Drive	OAuth 2.0	Shared drives, versioning, Docs export	P0

Dropbox	OAuth 2.0	Shared folders, file requests, selective sync	P0
OneDrive	OAuth 2.0 / Azure AD	SharePoint, shared libraries, naming restriction awareness	P0
Backblaze B2	Application Key	Lifecycle rules, S3-compatible mode	P0
LAN Direct	Discovery / manual	Peer-to-peer, zero-config where possible	P0
Server-to-Server	Inherited	Direct transfer without local routing	P0
Azure Blob	Conn String / Azure AD	Container mgmt, SAS tokens	P1
Google Cloud Storage	Service Account / OAuth	Bucket mgmt, lifecycle, IAM	P2

14.3 Explicit Transfer Scope

The platform must support:

- local folder to local folder
- local volume to local volume on the same computer
- internal drive to internal/external drive
- external drive to external drive through the host computer
- removable drive to removable drive
- local computer to another computer on the same LAN
- computer to computer over SMB/NFS/SFTP/FTP
- remote to local and local to remote
- local to server/cloud and server/cloud to local
- server to server and cloud to cloud where supported
- guided device-to-device migration workflows

14.4 Transfer Features

- visible transfer queue
- per-item and aggregate progress
- ETA and throughput
- pause/resume
- retries with configurable policy
- bandwidth throttling
- priority ordering
- connection health indicators
- persistent queue across restart
- resumable transfer
- post-transfer verification

14.5 Drive-to-Drive and Peer-Transfer Behavior

Same-computer and peer transfers must use the same core framework as server/cloud transfers: queueing, preflight, pause/resume, verification, retry, compatibility handling, conflict handling, summaries and logs.

14.6 Server-to-Server Direct Transfer

When supported and secure, transfer directly between remote endpoints. Fallback to local relay must be transparent.

14.7 Connection Management

- saved connections
- bookmarks per connection
- OAuth for cloud
- SSH key auth
- connection test
- per-connection defaults

14.8 Device Discovery

- local network device discovery
- discovery of shares and reachable peers
- save/reuse peer devices
- fallback to manual connection

14.9 Migration Workflows

Workflows for: old computer to new computer, old drive to new drive, laptop to desktop, desktop to external backup, external drive consolidation.

Including: source/destination selection, compatibility preflight, space analysis, duplicate/conflict preview, resumable execution, post-run summary, retry of failed items.

General-User Requirement

The product must include guided setup for OneDrive, Google Drive, Dropbox, local folders, external drives, SMB shares, peer computers, and SFTP.

15. Functional Requirements — Sync & State Engine

15.1 Sync Modes

- one-way sync
- two-way sync
- mirror sync
- versioned backup sync

15.2 Execution Modes

- manual sync
- scheduled sync
- watch-based real-time sync
- API/CLI-triggered sync

15.3 Dry-Run and Preview

Before execution, users must preview: new files, changed files, deletions, conflicts, compatibility translations, collision risks, path-length risks, provider restriction warnings.

15.4 Conflict Philosophy

- **General/consumer default:** create conflict copies
- **Enterprise default:** policy-driven

Policies: ask, source wins, destination wins, newest wins, create duplicate, skip, quarantine for approval.

15.5 Verification and Rollback

- checksum verification option
- fast compare and full compare modes
- snapshot metadata before destructive operations
- rollback where possible
- partial failure continuation
- resumable sync state

15.6 Sync Contexts

Sync must work across: same-computer folders, same-computer drives, local to external, local to network share, local to peer, local to cloud/server, server to server, cloud to cloud.

15.7 Sync Feature Priority

Feature	Description	Priority
One-Way & Two-Way	Mirror, update-only, bidirectional with conflict resolution	P0

Real-Time Watch	Filesystem watcher triggers immediate sync	P0
Scheduled Sync	Cron-like: hourly, daily, weekly, custom	P0
Conflict Resolution	7 policies: newest, source, keep both, skip, quarantine, manual queue	P0
Selective Sync	Include/exclude patterns, size limits, type filters	P0
Sync State DB	SQLite-backed hashes, mod times, history per pair	P0
Dry Run	Preview all changes with approval step	P0
Reporting	Per-run summary: added, modified, deleted, conflicts, errors	P0
Delta Sync	Transfer only changed file portions (rsync-like)	P1
Bandwidth Control	Per-pair limits and scheduling windows	P1
Versioned Backup	Keep N versions, configurable retention	P1

General-User Requirement

Creating a basic sync job must be available through a guided wizard in Simple mode.

16. Functional Requirements — Universal Naming Compatibility Engine

Key Differentiator

When source naming rules differ from destination, the system must detect problems before failure, auto-handle low-risk cases, ask only for high-risk, preserve original logical name, continue the wider job, keep a mapping, and show a clear summary.

16.1 Why This Engine Is Required

The product cannot assume one universal naming model. Windows has reserved names and invalid characters, OneDrive/SharePoint impose path limits, APFS can be case-sensitive, ext4 limits filenames to 255 bytes.

16.2 Required Domains

- invalid character translation
- reserved filename handling
- path-shortening strategies
- Unicode normalization
- case-sensitivity mismatch
- case-only collision handling
- trailing space/period handling
- cloud-specific naming profiles
- duplicate emergence after translation

16.3 Logical vs Physical Naming

- **Logical name:** original source-facing name
- **Physical name:** destination-safe stored name
- **Mapping record:** reversible relationship
- **Display rule:** UI shows logical, physical, or both

16.4 Intervention Tiers

Tier	Behavior	Examples	UX
Tier 1 — Auto-quiet	Low-risk, log and badge	Safe normalization, deterministic char translation	Badge on summary. No modal.
Tier 2 — Visible auto	Medium-risk, notify	Path shortening, reserved-name escape, duplicate suffix	Inline notification, non-blocking.

Tier 3 — Decision required	High-risk, block for approval	Destructive collision, ambiguous normalization, multi-source collapse	Modal or approval queue.
-----------------------------------	-------------------------------	---	--------------------------

16.5 Policy Profiles

Per-destination: Windows-NTFS, macOS-APFS-CI, APFS-CS, ext4, OneDrive, Google Drive, Dropbox, SMB-to-Windows, S3, strict-enterprise.

16.6 Mapping and Restoration

- local mapping database
- optional sidecar manifests
- optional extended metadata
- compatibility event logs
- restore-original-name operation

16.7 Edge Cases

Scenario	Problem	Solution
macOS→Windows	NFD accented chars cause file-not-found	Auto-convert NFD to NFC, store mapping
Linux→Windows	Filename contains : ? *	Replace with full-width Unicode, store mapping
Any→Windows	Path exceeds MAX_PATH (260)	Truncate with hash, or \\?\ prefix
Any→Windows	File named CON/NUL (reserved)	Prefix with underscore, store mapping
Any→OneDrive	400-char path limit	Apply OneDrive profile, warn near limit
Any→ext4	Filename exceeds 255 bytes	Truncate preserving extension + hash suffix
Case collision	Report.txt and report.txt collide	Rename second with suffix, Tier 2 notify
Trailing dots/spaces	Windows strips silently	Detect and warn before transfer
Emoji filenames	Older filesystems fail	Detect support, warn or transliterate
Cloud round-trip	Provider normalizes differently	Detect and compensate automatically
Multi-source collapse	Two files normalize to same name	Tier 3: require user decision

16.8 Engine Architecture

Component	Responsibility
Unicode Normalizer	NFC↔NFD conversion, Hangul jamo, combining diacriticals, zero-width joiners

Character Sanitizer	OS-illegal chars to safe full-width Unicode equivalents, reversible
Length Validator	Target OS path length enforcement, intelligent truncation with hash suffix
Reserved Name Handler	Windows reserved names (CON, PRN, NUL, COM1-9, LPT1-9) escaping
Collision Resolver	Sequential numbers or hashes for normalization-created duplicates
Reversibility Mapper	Mapping DB for round-trip fidelity, original name restoration
Profile Engine	Per-destination profile loading and rule chaining
Cloud Adapter	Provider-specific rules for OneDrive, GDrive, Dropbox, S3, B2

General-User UX

Simple mode: “Some file names are not supported here.” “We adjusted 8 names.” “2 files need your attention.” Details available in Advanced mode.

17. Functional Requirements — Governance & Enterprise Control Plane

17.1 RBAC

Roles: org owner, org admin, security/compliance admin, approver, operator, user, read-only auditor.

17.2 Policy Engine

Policies control: allowed connectors, approved destinations, encryption requirements, max transfer size, file-type restrictions, checksum requirements, naming strictness, sync conflict policies, deletion restrictions, AI usage rules, data/audit retention.

17.3 Approvals

Workflows for: external destination transfers, destructive syncs, policy exceptions, high-risk naming Tier 3 scenarios, sensitive file movement.

17.4 Audit Logs

Captures: login/logout, connection CRUD, transfer start/finish/failure, sync runs, naming translations, approvals, policy changes, admin actions, API/CLI actions.

17.5 Device Health Dashboard

Shows: client version, last seen, last sync, transfer failure patterns, policy compliance, AI enablement, connector health, update status.

17.6 Shared Workspaces

Enterprise mode: shared connections, shared sync templates, shared policy-backed profiles, shared automation templates.

18. Functional Requirements — Security, Privacy, and Encryption

18.1 Desktop Security

Signed binaries, secure update pathway, least-privilege design, isolated preview/render, secure secret storage, UI/Rust separation.

18.2 Credentials

OS-native keychain (macOS Keychain, Windows Credential Vault, Linux Secret Service) with AES-256-GCM encrypted vault fallback.

18.3 Encryption

Feature	Description
In-Transit	TLS 1.3 where available. SSH/SFTP inherent. FTP auto-upgrade to FTPS.
At-Rest	AES-256-GCM vaults. Encrypt locally before cloud upload.
Key Management	Master password + Argon2id KDF. Optional YubiKey/FIDO2. Key escrow for business.
Cryptomator	Native read/write of Cryptomator vaults.
Zero-Knowledge	Encrypt-before-upload: cloud providers never see plaintext.
Selective	Encrypt specific folders/patterns while leaving others unencrypted.
Enterprise Policy	Admins enforce encryption per connector/destination/file type.

18.4 Session Controls

Session revocation, device deauthorization, policy refresh, connector access revocation.

19. Functional Requirements — API and CLI

19.1 CLI

login/auth, connection CRUD, transfer/sync execution, dry-run preview, structured output (JSON/YAML), compatibility reports, status queries, machine-friendly exit codes.

19.2 API

Admin workflows, device health, audit export, approvals, policy configuration, automation, SIEM/enterprise integration.

20. Functional Requirements — AI Assistance & Automation Engine

20.1 Scope

AI assistant + AI automation. No unrestricted destructive autonomy.

20.2 Use Cases

- explain failed transfers in plain language
- summarize job outcomes
- suggest sync rules and exclusions
- suggest folder cleanup actions
- classify folders and risk patterns
- detect duplicate risk
- identify compatibility issues before execution
- turn natural-language intent into job templates

20.3 Safety Rules

- AI may recommend actions
- destructive actions require confirmation or policy authorization
- generated rules must be previewable
- AI outputs must be auditable

20.4 Privacy

Policy-controlled AI routing. Content analysis opt-in only. Org admins govern model routing and availability. Local inference compatibility preserved in architecture.

General-User Requirement

AI must simplify the product, not make it feel more technical. Default experience is guided help, not an AI control panel.

21. User Experience and Interaction Requirements

21.1 Information Architecture

Simple Mode

Files, Transfers, Sync, Cloud & Servers, Search, Favorites, Activity, Settings.

Advanced Mode Adds

Detailed connections, checksums, duplicate inspection, automation center, naming inspector, archive internals, detailed logs, scripting.

Admin Console

Dashboard, users/roles, devices, policies, approvals, audit, connectors, AI governance, billing.

21.2 Onboarding

First-run: Welcome → Choose style (Personal/Power/Work) → Connect locations → First action → Explain compat → Enter workspace.

21.3 Guided Flows

15 required wizards: connect OneDrive/GDrive/Dropbox, connect local sync pair, connect SMB, connect SFTP, start transfer, create sync job, resolve interruption, handle compat warning, restore names, export report, transfer to another computer, copy to external drive, move old→new drive, backup folder, migrate between computers.

21.4 Content Design Rules

All text: plain language, no unexplained acronyms, explain what/why/what app did/what user can do next.

Example

BAD: "Destination normalization conflict due to policy profile."

GOOD: "This location cannot store the file name exactly as it is now. The app can adjust the name so the transfer continues."

21.5 Confidence and Recovery

Visible progress, retry failed, undo where feasible, recent activity history, renamed/moved file history, clear original↔destination name link, one-click recovery.

21.6 Key UI Components

- **Sidebar:** Favorites, devices, cloud, recent, sync pairs. Collapsible.
- **Status Bar:** Path, item count, size, transfer count, sync status, disk space.
- **Transfer Panel:** Slide-up, mini/full modes.

- **Notification Center:** Non-intrusive, per-event customizable.
- **Command Palette:** Cmd+K/Ctrl+K fuzzy search for any action.
- **Accessibility:** WCAG 2.1 AA, keyboard nav, screen reader, high contrast, scalable fonts, reduced motion.

22. Key User Journeys

22.1 Local Folder to OneDrive with Incompatible Names

25. User drags folder to OneDrive.
26. Preflight scan runs.
27. App says some names not supported.
28. Low-risk auto-adjusted.
29. High-risk surfaced.
30. Transfer continues.
31. Summary shows transferred/adjusted/skipped/blocked.

22.2 macOS to Windows Share Two-Way Sync

32. Create sync pair.
33. Preflight warns case/naming.
34. App recommends safe settings.
35. Job runs.
36. Conflicts become copies (Simple) or policy-driven (Enterprise).

22.3 Enterprise-Governed External Transfer

37. Select external destination.
38. Policy check.
39. Approval required.
40. Request sent.
41. Status visible.
42. Audit logged.

22.4 AI-Assisted Cleanup

43. Open messy folder.
44. AI suggests duplicates/cleanup.
45. Preview actions.
46. Accept selected.
47. Execute and record.

22.5 Drive-to-Drive on Same Computer

48. Source in left pane, dest in right.

- 49. Drag across.
- 50. Preflight: space, compat, dupes, permissions.
- 51. Queue with progress.
- 52. Pause/resume/verify/summary.

22.6 Computer-to-Computer over LAN

- 53. Choose peer or discover.
- 54. Auth if needed.
- 55. Select folders.
- 56. Analyze bandwidth/space/compat.
- 57. Transfer with resume/verify.
- 58. Summary on both sides.

22.7 Computer Migration

- 59. Select “Migrate from another computer.”
- 60. Guide connection.
- 61. Select folders/drives/profile data.
- 62. Preview.
- 63. Handle compat automatically.
- 64. Summary + retry.

22.8 External Drive Backup

- 65. Select external drive.
- 66. Check space/compat/conflicts.
- 67. Choose copy/mirror/sync/versioned.
- 68. Queue with verify.
- 69. Summary: copied/skipped/adjusted/failed.

23. Non-Functional Requirements

23.1 Qualitative

Performance: responsive browsing, non-blocking UI, fast startup, efficient large-folder handling. Reliability: crash-safe queue, resumable jobs, sync durability, safe restart, strong error reporting. Portability: common core, destination-specific profiles. Accessibility: keyboard nav, screen reader, scalable fonts, contrast, reduced motion. Observability: job health telemetry, connector error patterns, compat incident rates, AI usage, device health.

23.2 Quantified Targets

Category	Requirement	Target
Performance	Cold start	<2 seconds
Performance	10K file listing	<500ms local, <2s remote
Performance	Concurrent streams	Up to 20/connection, 100 total
Performance	RAM idle	<100MB
Performance	RAM under load	<500MB
Performance	Transfer throughput	>90% theoretical bandwidth
Reliability	Resume after crash	100% for files >1MB
Reliability	Sync integrity	Zero data loss (checksums)
Reliability	Crash-free rate	>99.9%
Security	Credentials	OS keychain + encrypted fallback
Security	At-rest encryption	AES-256-GCM
Security	In-transit	TLS 1.3 minimum
Compatibility	macOS min	macOS 12 (Monterey)
Compatibility	Windows min	Windows 10 (21H2)
Compatibility	Linux min	Ubuntu 22.04, Fedora 38, Arch
Installer	Formats	.dmg, .msi+.exe, .deb+.ApplImage
Updates	Auto-update	Tauri updater, differential, rollback
Localization	Languages	EN, ES, FR, DE, PT, JA, ZH, KO
Telemetry	Analytics	Opt-in only, no PII

24. Success Metrics

24.1 Categories

Usability: first transfer rate, first sync rate, onboarding completion, time-to-first-task, Simple mode retention, help-open rate, peer/drive transfer rates, migration wizard completion.

Reliability: transfer/sync success, resume success, crash-free, mean recovery time.

Compatibility: auto-resolved %, jobs saved, manual decision %, name restoration rate.

Enterprise: managed devices, policy compliance, approval turnaround, audit completeness. AI: suggestion acceptance, failure reduction, conflict handling reduction, support contact reduction.

24.2 Quantified KPI Targets

Metric	6 Month	12 Month
Downloads	50,000	200,000
MAU	20,000	80,000
Business teams	100	500
MRR	\$10,000	\$60,000
App rating	4.3+	4.5+
Transfer success	99.5%+	99.9%+
Compat auto-resolve	95%+	99%+
Crash-free	99.8%+	99.9%+
Onboarding completion	70%+	85%+
AI suggestion acceptance	40%+	60%+
NPS	30+	50+
Migration wizard completion	60%+	80%+
Time to first task	<5 min	<3 min

25. Risks and Mitigations

Risk	Severity	Likelihood	Mitigation
Scope overload	High	High	Subsystem-based structure. Connector gates. Strict GA criteria.
UX overload	High	High	Simple mode default. Progressive disclosure. Plain-language writing.
Connector instability	Medium	High	Abstraction layer. Provider test matrix. Telemetry maintenance.
Compat edge cases	High	High	Deterministic translation. Per-dest profiles. Mapping store. Tier 3 escalation.
AI trust/privacy	Medium	Medium	Opt-in content analysis. Policy-controlled AI. Approval for destructive. Audit trail.
Network/peer variability	Medium	High	Fallback paths. Diagnostics. Guided setup. Resumability.
Tauri 2.0 stability	High	Medium	Pin stable. Engage Tauri team. Monitor releases.
Cross-platform UI issues	Medium	Medium	Per-platform visual regression in CI. Beta pool on all 3 OS.
Security vulnerabilities	Critical	Low	Third-party audit at RC. Bug bounty at launch. CVE scanning.
Low adoption	Medium	Medium	Free-for-individual. Market compat differentiator.

26. Engineering Decomposition

26.1 Rust Core Modules

- filesystem abstraction
- transfer engine
- sync engine
- compatibility engine
- connector adapters
- hashing and integrity
- metadata/index store
- secure vault
- scheduler/background workers
- audit/event emission
- local-peer discovery and peer transfer
- migration workflow orchestration

26.2 Desktop UI Modules

- browser shell
- panes and tabs
- transfer center
- sync designer
- search center
- connection setup
- compatibility summary UI
- guided wizards
- settings
- AI assistance layer
- drive migration wizard
- computer transfer wizard

26.3 Admin Console Modules

- auth and tenant management
- RBAC
- devices
- policies
- approvals
- audit explorer
- connector governance
- AI governance
- billing

27. Launch Definition

27.1 GA Requirements

v1 GA must include: Windows/macOS/Linux desktop app, admin console, all connectors, file manager, transfer engine, sync modes, compatibility engine, enterprise governance, API/CLI, AI, Simple/Advanced modes, drive-to-drive, computer-to-computer, migration workflows.

27.2 Internal Sequencing

- 70. local file manager core
- 71. transfer queue and connections
- 72. drive-to-drive and removable media
- 73. peer/computer-to-computer
- 74. sync engine
- 75. compatibility engine
- 76. Simple mode and onboarding
- 77. enterprise control plane
- 78. AI assistance and automation
- 79. full cross-platform certification

28. Development Timeline — 18–24 Months

Phase	Duration	Focus	Deliverable
Phase 1: Foundation	Months 1–3	Core architecture, Tauri+Rust+Vite, dual-pane file manager	Alpha on all 3 platforms
Phase 2: Transfer	Months 4–6	Transfer queue, SFTP/FTP/WebDAV/SMB/NFS, drive-to-drive	Beta 1: local + remote
Phase 3: Cloud + Peer	Months 7–9	S3/GDrive/Dropbox/OneDrive/B2, peer discovery, computer-to-computer	Beta 2: full cloud + peer
Phase 4: Compat + Sync	Months 10–12	Compatibility engine, sync engine, conflict resolution, migration	Beta 3: compat + sync
Phase 5: AI + Advanced	Months 13–15	AI assistant, terminal, encryption, CLI, performance	Feature Complete
Phase 6: Enterprise + UX	Months 16–18	Admin console, RBAC, policies, Simple mode, onboarding, API	Release Candidate
Phase 7: Launch	Months 19–21	Certification, installers, docs, bug bash	v1.0 GA
Phase 8: Stabilize	Months 22–24	Post-launch fixes, optimization, community feedback	v1.1 Stability

28.2 Milestones

Milestone	Month	Gate Criteria
Alpha	3	File manager on all 3 platforms. 80%+ test coverage.
Beta 1	6	All transfer protocols. Queue operational. Drive-to-drive working.
Beta 2	9	All cloud providers. Peer transfer. <500ms start. <100MB RAM.
Beta 3	12	Compat engine 200+ tests. Sync 10K+ files. Migration wizard.
Feature Complete	15	All epics code-complete. No P0 bugs. Security audit started.
Release Candidate	18	<50 bugs (no P0/P1). Security audit passed. Enterprise operational.
v1.0 GA	21	Docs complete. Installers on all platforms. Support operational.
v1.1	24	Top issues resolved. Performance optimized. v1.5 roadmap published.

29. Team Composition

Role	Count	Responsibility	Phase
Lead Rust Eng / Architect	1	Core architecture, file ops, transfer, sync, compat ownership	1
Rust Systems Engineers	2	Protocols, cloud connectors, encryption, peer, migration	1–2
Rust Backend Engineer	1	CLI, API, audit, scheduler, SQLite	2
Frontend Engineers (Desktop)	2	Dual-pane, transfer center, wizards, terminal, responsive	1
Frontend Eng (Admin Console)	1	Admin console: RBAC, policies, audit, billing	5
AI/ML Engineer	1	AI integration, prompts, suggestions, local inference arch	4
UX/UI Designer	1–2	Design system, research, usability, Simple/Advanced, a11y	1
UX Writer	0.5	Error messages, wizard copy, compat explanations	3
QA Engineers	1–2	Cross-platform, E2E, compat edge cases (200+ tests), perf	2
DevOps	1	CI/CD, auto-update, crash reporting, telemetry	1
Security Engineer	0.5	Audit coordination, threat model, crypto review	4
Product Manager	1	Roadmap, user feedback, competitive analysis, GTM	1
Technical Writer	0.5	User docs, API docs, CLI reference, admin guide	6

Total: 13–17 people, ramping from ~6 in Phase 1 to full team by Phase 4.

30. Final Product Definition

This product is a consumer-simple, power-user fast, enterprise-governed file operations platform for Windows, macOS, and Linux.

It combines: premium desktop file management, multi-protocol and multi-cloud transfer, resilient synchronization, compatibility-aware naming and path handling, explicit drive-to-drive and computer-to-computer workflows, enterprise governance, and AI assistance and automation.

Its defining promise is not that every destination natively accepts every filename. Its promise is that the platform handles those differences intelligently, keeps work moving, preserves logical identity, and explains what happened clearly.

31. External Constraints

- ForkLift validates the combined file-manager-plus-transfer model.
- Tauri supports React+Vite desktop frontends natively (recommended scaffold).
- Windows naming rules vary by filesystem and include reserved conventions.
- APFS can be case-sensitive.
- ext4 has a 255-byte filename limit.
- OneDrive/SharePoint enforce 400-character path limits and naming restrictions.

32. Post-v1 Roadmap

Feature	Description	Target
Mobile Companion	iOS/Android for browsing and triggering sync (Tauri 2.0 mobile)	v1.5
Peer-to-Peer (QUIC)	Direct device-to-device without LAN intermediary	v1.5
Plugin System	WASM-based sandboxed runtime with marketplace	v2.0
AI Organization	ML-based categorization, duplicate detection, smart folders	v2.0
Team Workspaces	Shared connection profiles, sync configs, transfer templates	v2.0
Git Awareness	Native status indicators, .gitignore respect, branch-aware ops	v1.5
Virtual Drive Mount	Mount remote/cloud as native drives (FUSE/WinFsp)	v2.0
Automated Workflows	If-this-then-that: file arrives → encrypt and upload	v2.0
Extended REST API	Full local API for programmatic control	v1.5
Local AI Inference	On-device AI for privacy-sensitive environments	v2.0

Appendix A: Glossary

Term	Definition
NFC	Unicode Normalization Form Composed (Windows default)
NFD	Unicode Normalization Form Decomposed (macOS default)
FUSE	Filesystem in Userspace (Linux/macOS virtual filesystems)
WinFsp	Windows File System Proxy (Windows FUSE equivalent)
WASM	WebAssembly (portable binary for sandboxed execution)
PTY	Pseudo-terminal (virtual terminal for embedded shell)
IPC	Inter-Process Communication (Tauri bridge: Rust↔React)
MAX_PATH	Windows 260-char path limit (extendable to 32,767)
RBAC	Role-Based Access Control
SAML	Security Assertion Markup Language (SSO standard)
SCIM	System for Cross-domain Identity Management (provisioning)
SIEM	Security Information and Event Management
E2E	End-to-end encryption
Delta Sync	Transferring only changed file portions
Logical Name	Original source filename in compatibility mapping
Physical Name	Destination-safe filename complying with target rules
Compatibility Profile	Per-destination rule set (chars, paths, case, Unicode)
Intervention Tier	Compat issue severity (Tier 1: auto, Tier 2: visible, Tier 3: decision)

— End of Document —